

Theory

Question 1.

(a)

1. Direct Sampling

- **Strengths:** Simple to implement; straightforward when the probability distribution is fully known.
- **Weaknesses:** Inefficient when rare events need to be sampled or when dealing with highly conditional distributions, like 0.015 probability that a person prefers to travel by bus and has a low stress level.
- **Application:** For the dataset, direct sampling would involve generating samples proportional to the given probabilities, like selecting 80% for air/train preference directly from the data.

2. Rejection Sampling

- **Strengths:** Useful when the target distribution is difficult to sample directly; ensures samples meet specific conditions.
- **Weaknesses:** Inefficient when the acceptance rate is low, leading to computational overhead.
- **Application:** To estimate rare combinations like "bus preference and low stress with 0.015 probability" rejection sampling would filter out unlikely cases.

3. Gibbs Sampling

- **Strengths:** Ideal for high-dimensional distributions and conditional dependencies; ensures convergence over time.
- **Weaknesses:** Requires careful initialization and might take many iterations to converge to the true distribution.
- **Application:** Useful for iteratively estimating probabilities involving dependencies, e.g., stress levels conditional on travel preferences.

(b)

- Given:
 $P(\text{Leisure} \mid \text{Train}) = 0.400$
Train preferences in sampling of 100 peoples = 30, $P(\text{Train}) = 0.3$
- Expected number of leisure travelers:
Expected count = $P(\text{Leisure} \mid \text{Train}) * P(\text{Train}) * \text{total_samples} = 0.400 * 30 * 100 = 12$

You would expect 12 people out of the 30 train travelers to travel for leisure.

(c)

- Given:
 $P(\text{Air travel}) = 0.800$
 $P(\text{Business} \mid \text{Air travel}) = 0.200$.
- Joint probability:
 $P(\text{Air travel}, \text{Business}) = P(\text{Air travel}) * P(\text{Business} \mid \text{Air travel}) = 0.800 * 0.200 = 0.160$.

Probability that a randomly selected person prefers air travel and travels for business 0.160

(d)

- **Accuracy:**
Accuracy measures how close the estimated probabilities are to the true values. With a larger sample size, random sampling errors reduce, leading to more accurate probability estimates.
- **Precision:**
Precision refers to the variability of the estimates. Larger sample sizes decrease the standard error, reducing the variability, effect of outliers and improving confidence in the estimates (closer to true probability).
- **Implication:**
Estimating rare probabilities, eg. $P(\text{Bus preference and low stress}) = 0.015$, becomes more reliable as more samples are collected and for common probabilities, eg. $P(\text{air} \vee \text{train}) = 0.8$, larger sample sizes improve precision, ensuring consistent estimates across repeated samplings.

Question 2.

a) Identifying Random Variables and Representing Statements in Symbols

Random Variables:

- B: Person reads books.
- J: Person accesses academic journals.
- C: Person participates in book clubs.

Statements in symbolic form:

1. $P(B \vee J) = 0.91$
2. $P(J | B) = 0.40$, $P(\neg J | B) = 0.60$
3. $P(C | B) = 0.320$
4. $P(J \wedge \neg B) = 0.227$
5. $P(\neg B \wedge \neg J) = 0.090$
6. $P(J | \neg B) = 0.716$
7. $P(C \wedge J) = 0.088$
8. $P(C \vee J) = 0.631$
9. $P(J | C) = 0.40$
10. $P(J) = 0.50$
11. $P(C | \neg B) = 0.0044$

(b) Verifying Validity of the Probability Distribution

To confirm validity, the probabilities must satisfy these **axioms**:

1. $0 \leq P(X) \leq 1$ for any event X.
2. $P(S) = 1$, S is the sample space.
3. $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

Check:

- Axiom 1 is satisfied for all 11 cases
- Total probability : $P(S) = P(B \vee J) + P(\neg B \wedge \neg J) = 0.91 + 0.09 = 1$, satisfying axiom 2.
- Conditional probabilities : $P(J | B) = 0.40$, $P(\neg J | B) = 0.60$ sum to 1.

Hence, the propositions create a valid probability distribution.

(c) Joint Probability Distribution Table

- $P(B \wedge J \wedge C) = 0.088$
- $P(B \wedge J \wedge \neg C) = 0.185$
- $P(B \wedge \neg J \wedge C) = 0.131$
- $P(B \wedge \neg J \wedge \neg C) = 0.279$
- $P(\neg B \wedge J \wedge C) = 0.00064$
- $P(\neg B \wedge J \wedge \neg C) = 0.22636$
- $P(\neg B \wedge \neg J \wedge C) = 0.000396$
- $P(\neg B \wedge \neg J \wedge \neg C) = 0.09$

(d) Check Conditional Independence

Two random variables X and Y are **conditionally independent** Z if:

$$P(X \wedge Y | Z) = P(X | Z) \cdot P(Y | Z)$$

I. B and J given C

$$P(B \wedge J | C) = P(B | C) \cdot P(J | C)$$
$$P(B \wedge J | C) = \frac{P(B \wedge J \wedge C)}{P(C)} = 0.4$$
$$P(B | C) = \frac{P(B \wedge C)}{P(C)} = 0.995$$
$$P(J | C) = \frac{P(J \wedge C)}{P(C)} = 0.403$$
$$P(B | C) \cdot P(J | C) = 0.401$$

whic close to $P(B \wedge J | C)$

$\therefore$ B and J are conditionally independent on C

II B and C given J

$$P(B \cap C | J) = \frac{P(B \cap C \cap J)}{P(J)} = 0.176$$

$$P(B | J) = \frac{P(B \cap J)}{P(J)} = 0.576$$

$$P(C | J) = \frac{P(C \cap J)}{P(J)} = 0.177$$

$$P(B | J) \cdot P(C | J) = 0.096 \neq 0.176$$

$\therefore$

$$P(B \cap C | J) \neq P(B | J) P(C | J)$$

$\therefore$ B and C are not conditionally independent on J

III J and C given B

$$P(J \cap C | B) = \frac{P(J \cap C \cap B)}{P(B)} = 0.129$$

$$P(J | B) = \frac{P(J \cap B)}{P(B)} = 0.4$$

$$P(C | B) = \frac{P(C \cap B)}{P(B)} = 0.320$$

$$P(J | B) \cdot P(C | B) = 0.128$$

$$\therefore P(J \cap C | B) \approx P(C | B) \cdot P(J | B)$$

$\therefore$ They are independent

Question 3.

(a) Bayesian Inference

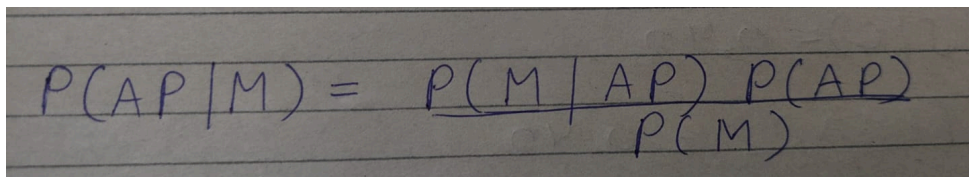
Let the problem involve two possible causes of a misclassification alarm:

1. AP: The misclassification is caused by adversarial perturbations.
2. BA: The misclassification is caused by a backdoor attack.
3. M: The misclassification alarm is raised

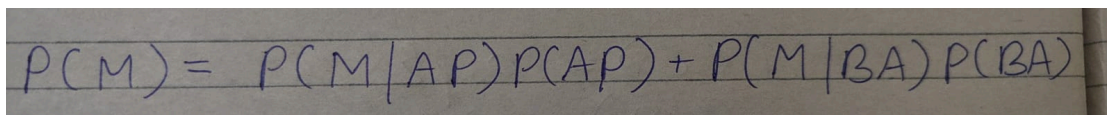
Initially, AP and BA are considered independent, $P(AP, BA) = P(AP) \cdot P(BA)$, but new information suggests an increased prevalence of backdoor attacks $P(BA)$ is higher.

We want to update the belief about $P(AP | M)$, the probability that adversarial perturbations caused the alarm given that a misclassification alarm occurred, after learning that backdoor attacks are more prevalent.

Using **Bayes' theorem**:


$$P(AP | M) = \frac{P(M | AP) P(AP)}{P(M)}$$

Where **$P(M|I)$ is total probability**


$$P(M) = P(M | AP) P(AP) + P(M | BA) P(BA)$$

adversarial perturbations and backdoor attacks are independent initially, but the observation of M links them

(b) Probabilities Involved

- **Prior:**
 - $P(AP)$: Initial probability of adversarial perturbations causing the misclassification (prior to any alarm or new information).
 - $P(BA)$: Initial probability of a backdoor attack causing the misclassification (prior to any alarm or new information).
- **Likelihood:**
 - $P(M|AP)$: Probability of a misclassification alarm given adversarial perturbations.
 - $P(M|BA)$: Probability of a misclassification alarm given a backdoor attack.
- **Posterior:**
 - $P(AP|M)$: Probability that adversarial perturbations caused the alarm after observing M.
 - $P(BA|M)$: Probability that backdoor attack caused the alarm after observing M.

(c) Impact of Conditioning on Backdoor Prevalence

Observations:

1. Higher $P(BA)$ implies that backdoor attacks are now more likely a cause of M, increasing $P(M|B) \cdot P(BA)$
2. Since M is observed, the increased $P(BA)$ reduces the contribution of $P(AP)$ to $P(M)$, thus lowering $P(A|M)$.

When two independent causes AP and BA contribute to the same observed effect M, an increase in the likelihood of one cause BA reduces the likelihood of the other AP as an explanation for M. This is because the observed effect can now be sufficiently explained by the more likely cause BA.

The updated belief $P(AP|M)$ decreases due to the increased likelihood of backdoor attacks being the primary cause of M, because posteriors are complementary.

Coding

4. Bayesian network for fare classification

Task 1: Initial Bayesian Network Construction

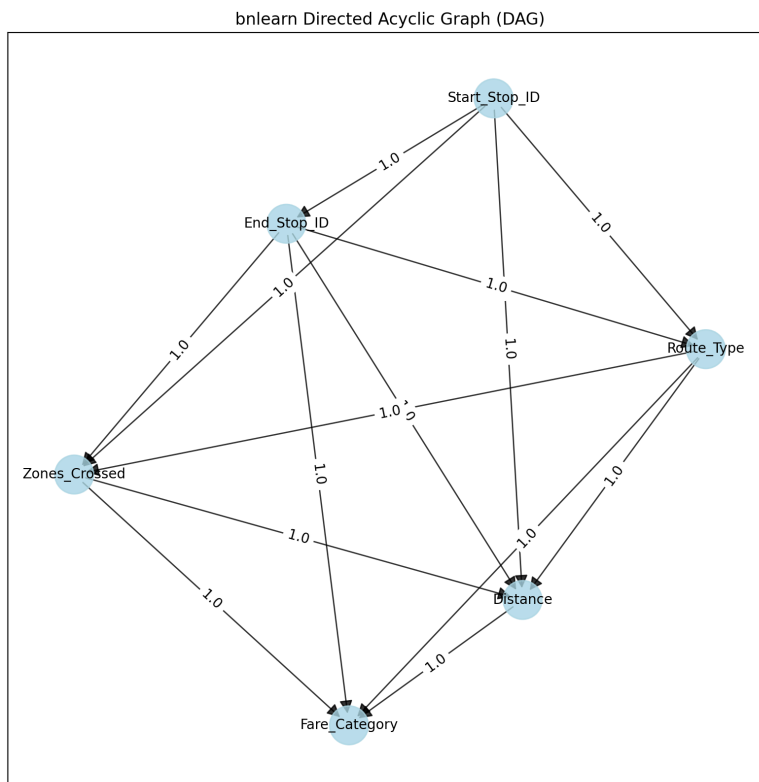
1. Model Description:

- The initial network includes dependencies between almost all possible feature pairs. (one edge was removed otherwise program is crashing because of high memory usage)
- The structure is defined manually as a directed acyclic graph (DAG).

2. Implementation:

- Used the **bnlearn** library to create and fit the network.
- Encoded categorical variables using predefined mappings:
 - **Distance**: {"short": 1, "medium": 2, "long": 3}
 - **Fare Category**: {"Low": 1, "Medium": 2, "High": 3}

3. Visualization: The network structure contains all possible edges representing dependencies between features.



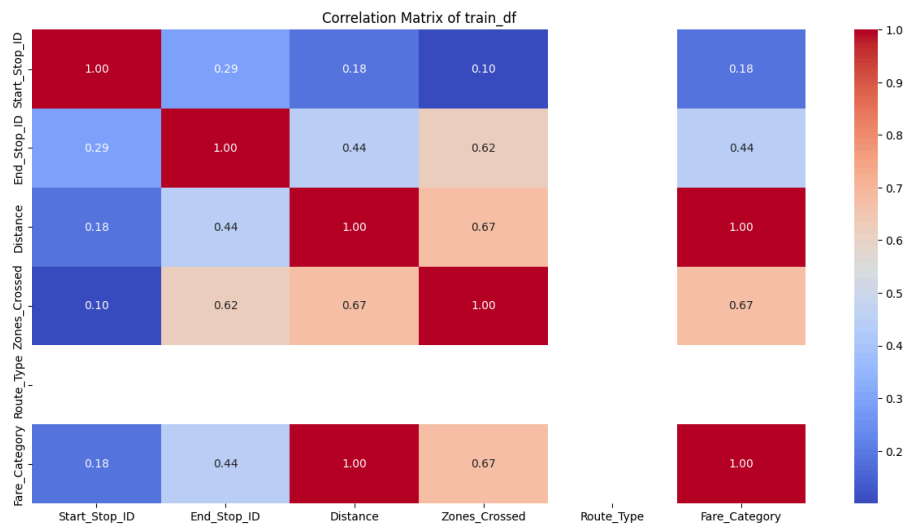
4. Evaluation:

- Runtime for training: **242.45 seconds**
- Model Size : **51,846 KB**
- Validation accuracy: **100%**

Task 2: Pruned Bayesian Network

1. Pruning Methodology:

- Edges with correlations below a threshold of 0.4 were removed.
- Correlation matrix was computed using the training dataset.
- Pruned edges were identified and excluded from the DAG.



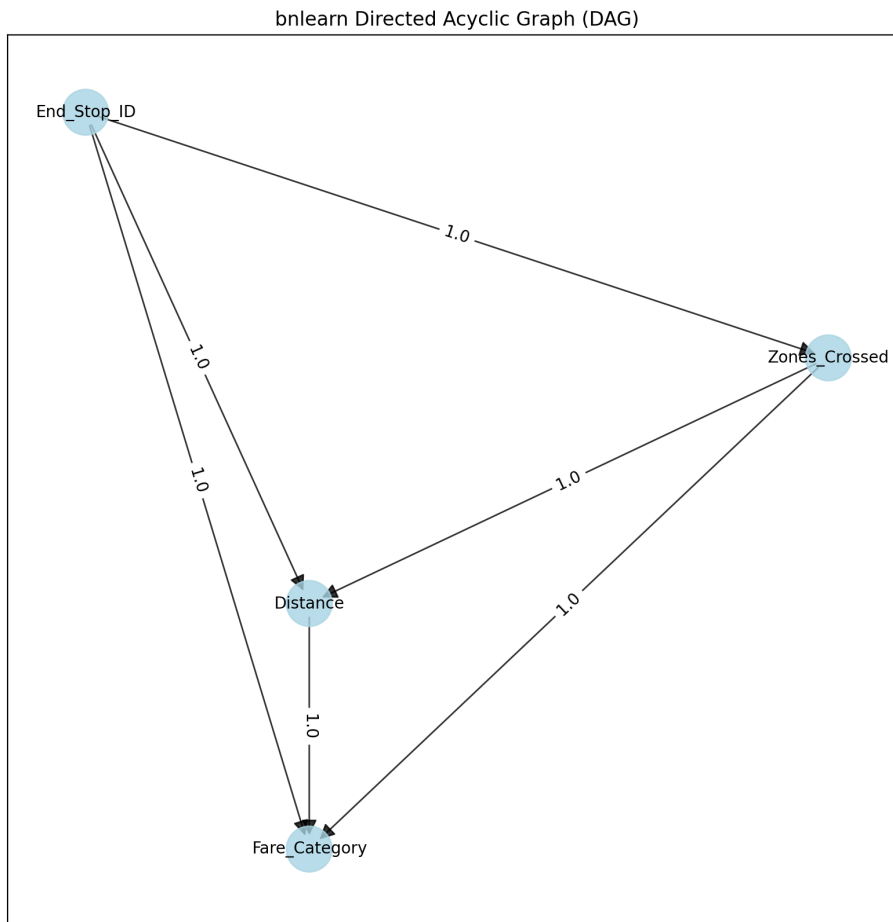
2. Edges Removed:

('Start_Stop_ID', 'Fare_Category'), ('Route_Type', 'Fare_Category'), ('Start_Stop_ID', 'Distance'), ('Route_Type', 'Distance'), ('Start_Stop_ID', 'Zones_Crossed'), ('Route_Type', 'Zones_Crossed'), ('Start_Stop_ID', 'Route_Type'), ('End_Stop_ID', 'Route_Type'), ('Start_Stop_ID', 'End_Stop_ID')]

3. Benefits of Pruning:

- Reduced model complexity.
- Improved runtime efficiency without loss in accuracy.

4. **Visualization:** The pruned network shows fewer edges, focusing only on highly correlated relationships.



5. **Evaluation:**

- Runtime for training: **14.42 seconds**
- Model Size : **1,581 KB**
- Validation accuracy: **100%**

Task 3: Optimized Bayesian Network

1. **Optimization Techniques:**

- Applied structure learning using the Hill Climbing algorithm.
- Used **bnlearn's structure_learning.fit** with a whitelist of allowed edges.
- Refined network structure to improve accuracy and efficiency.

2. **Optimization Results:**

- The optimized structure reduced unnecessary connections while introducing potentially better ones based on data patterns.

- Conditional Probability Tables (CPTs) were recomputed for the refined structure.
3. **Visualization:** The optimized network shows a refined structure learned directly from the dataset.



4. **Evaluation:**
- Runtime for training: **0.64 seconds**
 - Model Size : **16 KB**
 - Validation accuracy: **100%**

5. Tracking a Roomba Using the Viterbi Algorithm

Implementation and application of the Viterbi algorithm to estimate a Roomba's path on a grid given noisy observations. The algorithm leverages the principles of Hidden Markov Models (HMMs) to compute the most probable sequence of states. The implementation accounts for observation noise and employs two movement policies (`random_walk` and `straight_until_obstacle`) to analyze their impact on accuracy.

Problem Modeling

The Roomba tracking problem is modeled as a Hidden Markov Model (HMM):

1. State Space:

- Each state is defined as $S_t = (\text{position}, \text{heading})$:
 - `position` is a tuple (x, y) representing the Roomba's location on the grid.
 - `heading` is one of $\{\text{North (N)}, \text{East (E)}, \text{South (S)}, \text{West (W)}\}$.

2. Transition Probabilities:

- Random Walk Policy:**
 - The Roomba transitions uniformly to any valid next state. The probabilities are evenly distributed among all possible moves from the current position.
- Straight Until Obstacle Policy:**
 - The Roomba moves deterministically until it encounters an obstacle, at which point it chooses a new heading uniformly.

3. Emission Probabilities:

- The observed position is modeled as a Gaussian distribution centered at the true position with a standard deviation $\sigma = 1.0$. The probability of observing (obs_x, obs_y) given the true state is

$$P(O_t|S_t) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x - obs_x)^2 + (y - obs_y)^2}{2\sigma^2}\right)$$

4. Viterbi Algorithm:

- The Viterbi algorithm is a dynamic programming method used to find the most likely sequence of states in a Hidden Markov Model (HMM), given a sequence of observed events. Here's a step-by-step explanation:

Inputs

1. **States (S):** $S = \{s_1, s_2, \dots, s_N\}$.
2. **Observations (O):** Sequence $O = \{o_1, o_2, \dots, o_T\}$.
3. **Start State (s_0):** Known state at $t = 0$.
4. **Transition Probabilities (A):** $A[i][j] = P(s_j | s_i)$.
5. **Emission Probabilities (B):** $B[j][o_t] = P(o_t | s_j)$.

Algorithm Steps

1. **Initialization (Time $t = 1$):** Since the start state s_0 is known, initialize:

$$V_1(s_j) = A[s_0][s_j] \cdot B[j][o_1]$$

This is the probability of transitioning from s_0 to s_j and observing o_1 .

Store the path leading to each state:

$$\text{path}[s_j] = [s_0, s_j]$$

2. **Recursion (Time $t = 2, \dots, T$):** For each state s_j at time t , compute the maximum probability of reaching s_j :

$$V_t(s_j) = \max_{s_i \in S} (V_{t-1}(s_i) \cdot A[i][j] \cdot B[j][o_t])$$

Track the path:

$$\text{path}[s_j] = \text{path}[s_i] + [s_j]$$

where s_i is the state that maximized $V_t(s_j)$.

3. **Termination (Time $t = T$):** Find the state s^* with the highest probability at the final time step:

$$P^* = \max_{s_j \in S} V_T(s_j)$$

The most likely path corresponds to the path leading to s^* :

$$\text{Optimal Path} = \text{path}[s^*]$$

Evaluation

1. seed : 7

SEED: 7

Processing policy: random_walk

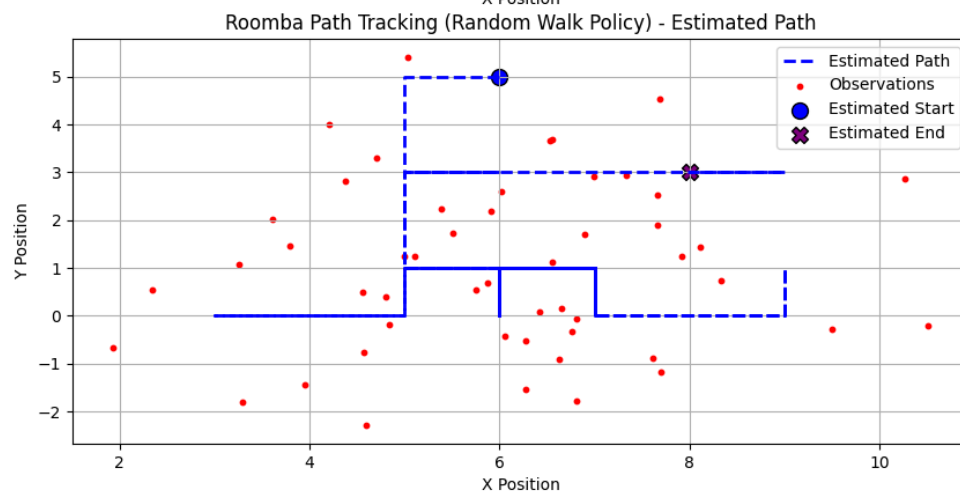
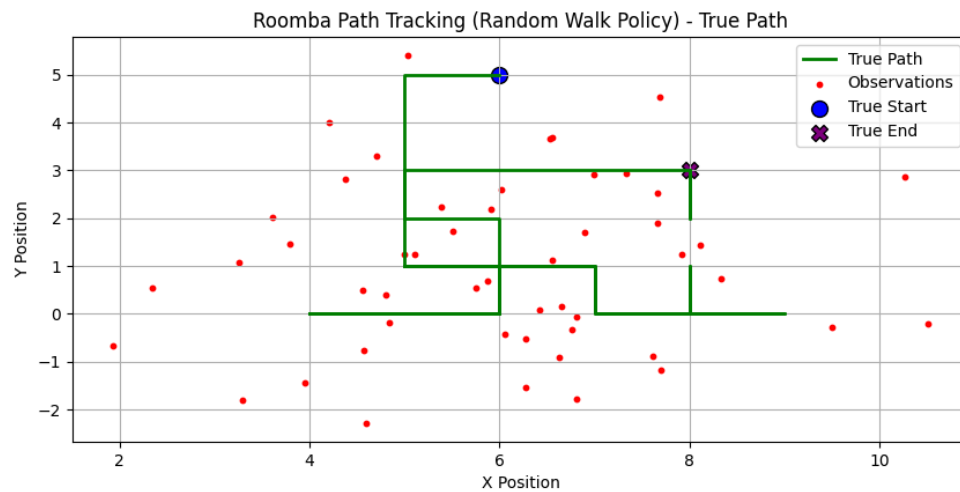
Viterbi Algorithm forward messaging: 100% | 48/48 [00:40<00:00, 1.18it/s]

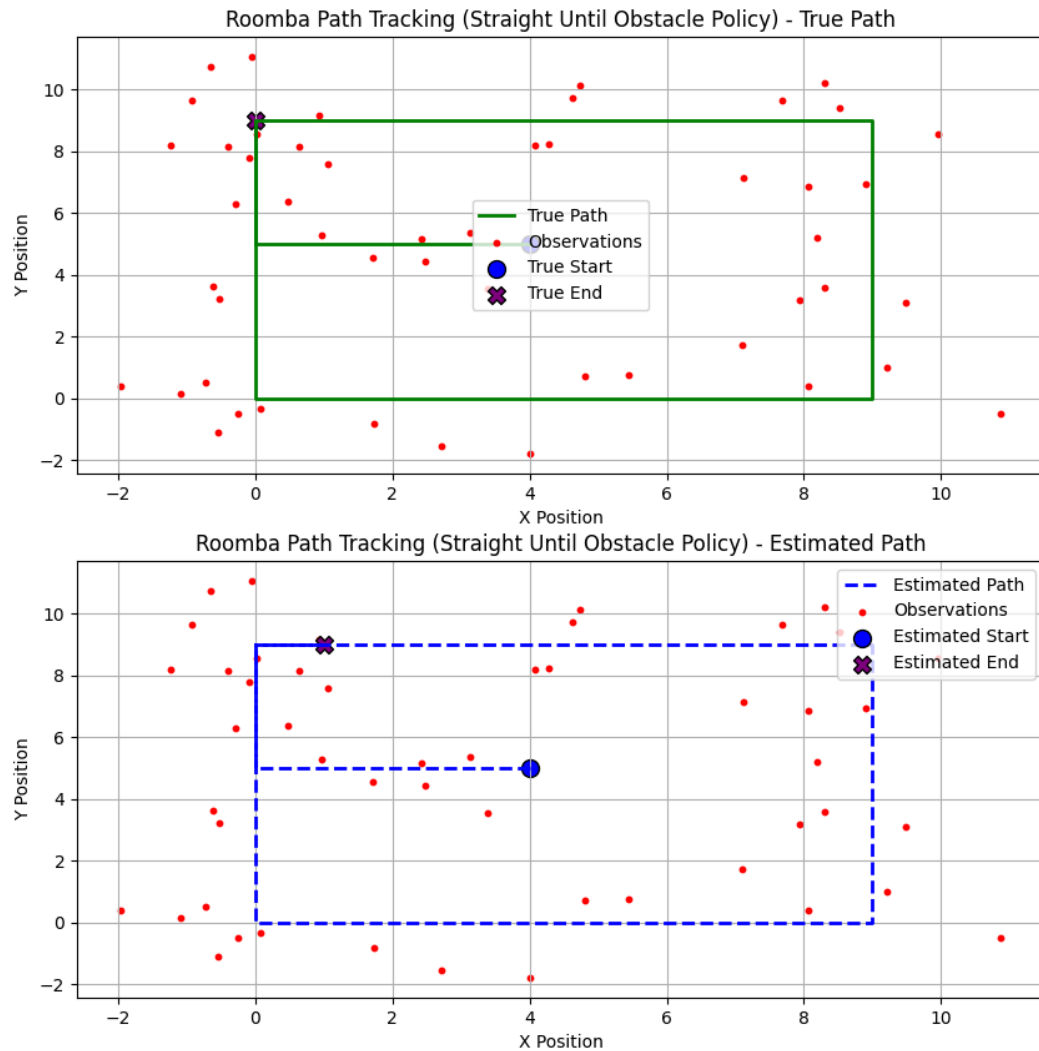
Tracking accuracy for random walk policy: 52.00%

Processing policy: straight_until_obstacle

Viterbi Algorithm forward messaging: 100% | 48/48 [00:29<00:00, 1.60it/s]

Tracking accuracy for straight until obstacle policy: 80.00%





2. seed : 10

SEED: 10

Processing policy: random_walk

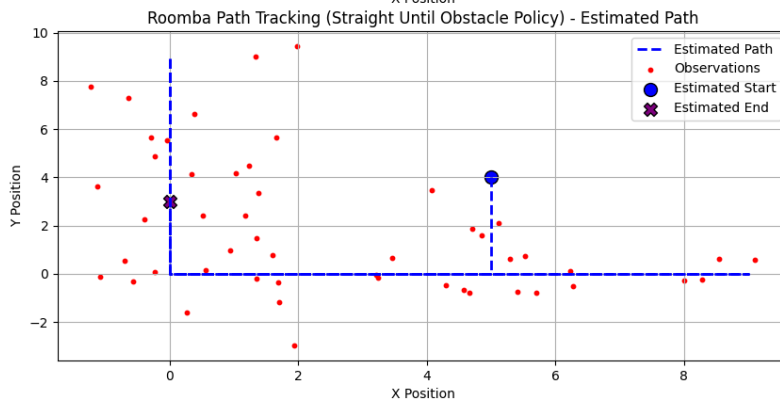
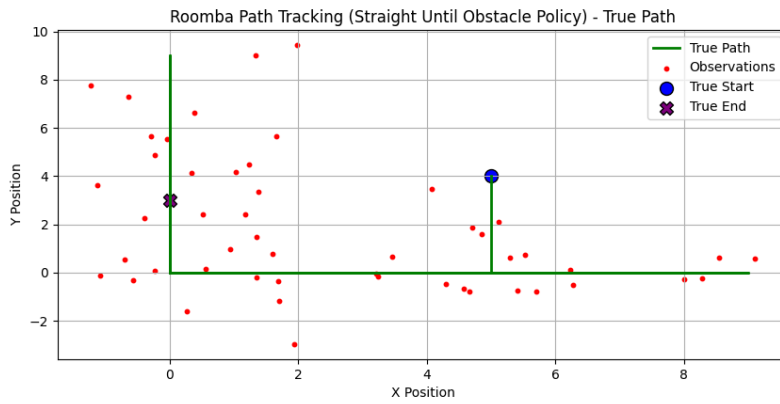
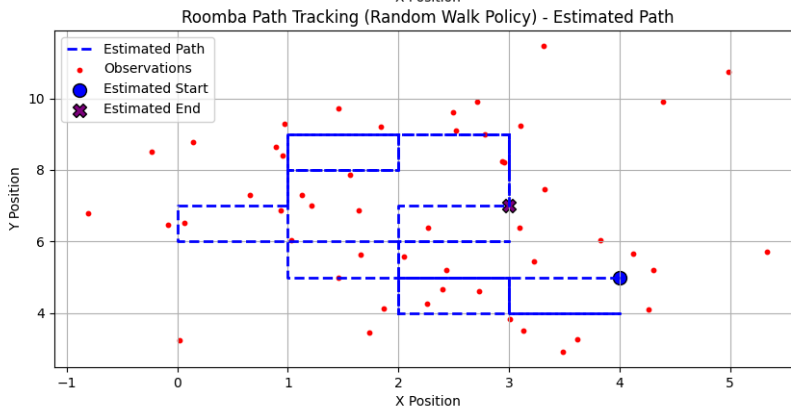
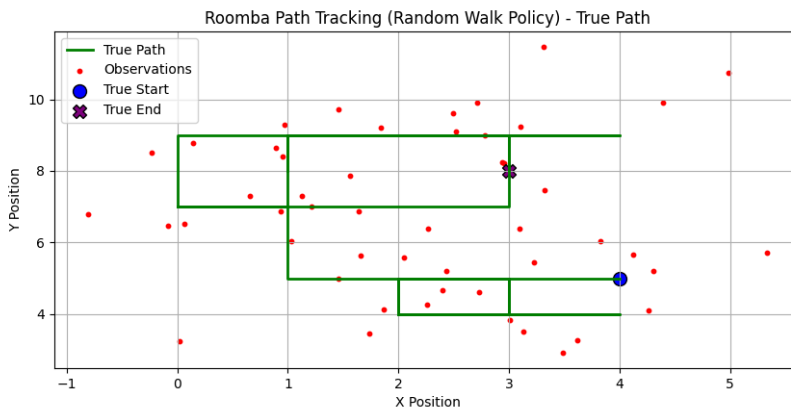
Viterbi Algorithm forward messaging: 100% | 48/48 [00:40<00:00, 1.18it/s]

Tracking accuracy for random walk policy: 60.00%

Processing policy: straight_until_obstacle

Viterbi Algorithm forward messaging: 100% | 48/48 [00:30<00:00, 1.59it/s]

Tracking accuracy for straight until obstacle policy: 100.00%



3. seed : 2004

SEED: 2004

Processing policy: random_walk

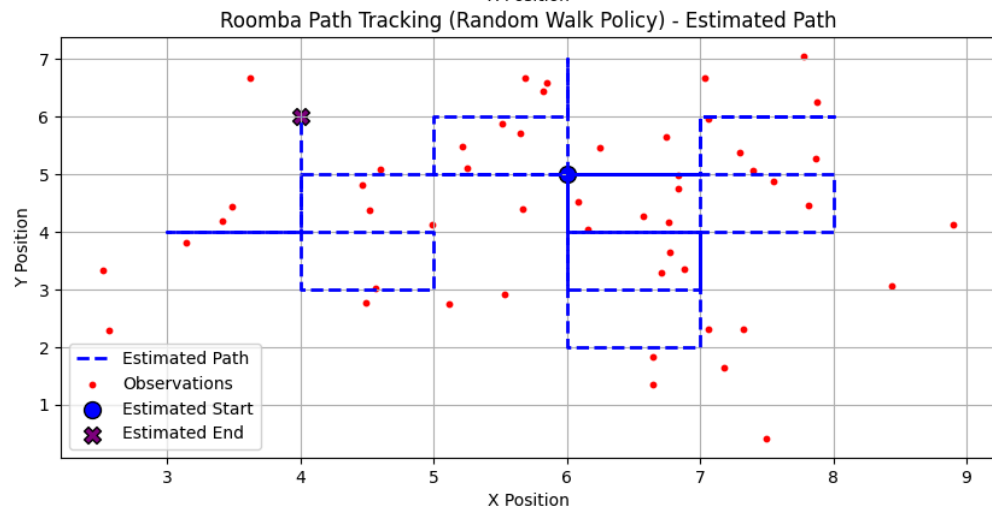
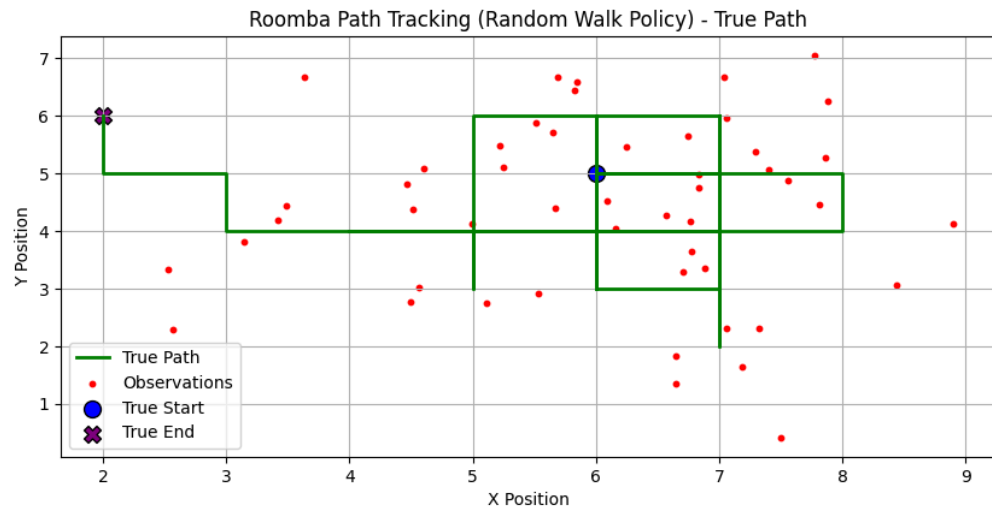
Viterbi Algorithm forward messaging: 100% | 48/48 [00:38<00:00, 1.26it/s]

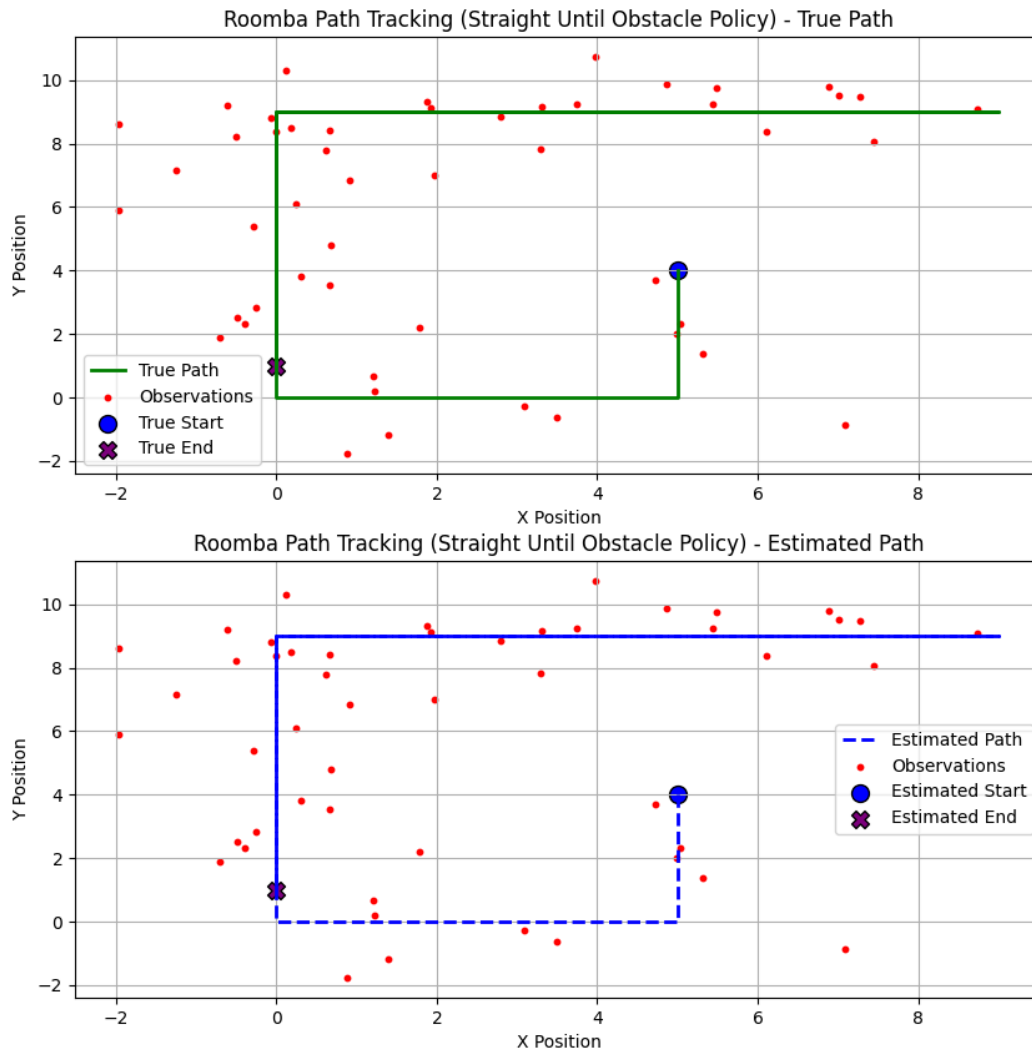
Tracking accuracy for random walk policy: 48.00%

Processing policy: straight_until_obstacle

Viterbi Algorithm forward messaging: 100% | 48/48 [00:32<00:00, 1.50it/s]

Tracking accuracy for straight until obstacle policy: 100.00%





Conclusion

The Viterbi algorithm successfully estimates the Roomba's path based on noisy observations. The **Straight Until Obstacle Policy** is more accurate in all cases due to its deterministic nature, which reduces uncertainty in transitions. In contrast, the **Random Walk Policy** has lower accuracy due to the stochastic nature of movements.